

Extending AALpy with Passive Learning: A Generalized State-Merging Approach

Benjamin von Berg¹(✉)[0009–0001–3595–4715] and
Bernhard K. Aichernig^{1,2}[0000–0002–3484–5584]



¹ Institute of Software Engineering and Artificial Intelligence,
Graz University of Technology, Austria

`benjamin.vonberg@tugraz.at`

² Institute for Formal Models and Verification,
Johannes Kepler University Linz, Austria

`bernhard.aichernig@jku.at`



Abstract. AALpy is a well-established open-source automata learning library written in Python with a focus on active learning of systems with IO behavior. It provides a wide range of state-of-the-art algorithms for different automaton types ranging from fully deterministic to probabilistic automata. In this work, we present the recent addition of a generalized implementation of an important method from the domain of passive automata learning: state-merging in the red-blue framework. Using a common internal representation for different automaton types allows for a general and highly configurable implementation of the red-blue framework. We describe how to define and execute state-merging algorithms using AALpy, which reduces the implementation effort for state-merging algorithms mainly to the definition of compatibility criteria and scoring. This aids the implementation of both existing and novel algorithms. In particular, defining some existing state-merging algorithms from the literature with AALpy only takes a few lines of code.

Keywords: automata learning · model inference · passive learning · state merging · red-blue framework

1 Introduction

AALpy [18] is an active automata learning library written in Python with a focus on systems that show IO behavior, i.e. reactive systems. Automata learning algorithms extract a finite-state model from observations of a system-under-learning. They form the basis for black-box checking [2, 21].

AALpy is open-source and published on github³. It is well-established in the community and has been used in various domains. This includes reinforcement learning [26, 27], analysis of communication protocols [22, 23] and neural networks [19], black-box checking [14, 25], anomaly detection [17], software testing [9], process mining [5, 13] and reuse in other learning algorithms [3, 12].

³ <https://github.com/DES-Lab/AALpy>

Passive learning algorithms use existing training data, like system logs, while active learning queries the system during learning. So far, AALpy has been mainly concerned with active automata learning. In this paper, we extend AALpy with a passive automata learning library for state-merging algorithms based on the red-blue framework [15]. This encompasses (1) algorithms already implemented in AALpy, such as RPNI [20] and IOAlergia [16], (2) existing algorithms that were not originally designed for the IO setting, such as the likelihood-ratio method by Verwer et al. [29] or EDSM [15] and (3) new approaches based on state-merging, be it completely novel methods or adaptations of existing methods in order to account for domain knowledge, such as additional constraints.

The rest of the paper is structured as follows. Section 2 gives some background and discusses how we generalized state merging. Section 3 demonstrates the usage of the new library. Section 4 draws the conclusions, relates AALpy to other tools and gives an outlook of further possible extensions. Further examples are provided in the Appendix.

2 Principles of Operation

2.1 Universal Internal Formalism

The tool presented in this work covers several types of finite-state automata with IO behavior. These automaton types share a small set of common assumptions: (1) There is a unique initial state from which execution starts. (2) Applying an input to an automaton triggers a state change and the emission of an output. (3) For each transition, the reached state is uniquely determined by the combination of current state, applied input and observed output. The following definition of automata covers Assumption 1 and 2 above and can be extended to more specific types of automata.

Definition 1. An *IO automaton* is a tuple $\mathcal{M} = \langle Q, q_0, I, O, T \rangle$, where Q is the set of states, $q_0 \in Q$ is the initial state, I and O are the sets of input and output symbols respectively and $T \subseteq Q \times I \times O \times Q$ is a set of transitions.

A transition $\langle q, i, o, q' \rangle \in T$ implies that applying input i in state q may cause $\mathcal{M}$ to emit the output o and switch to state q' . In other words, the automaton might behave nondeterministically. We discern between different forms of determinism.

Definition 2. An automaton $\mathcal{M} = \langle Q, q_0, I, O, T \rangle$ is *deterministic* iff

$$\forall \langle q_1, i_1, o_1, q'_1 \rangle, \langle q_2, i_2, o_2, q'_2 \rangle \in T : q_1 = q_2 \wedge i_1 = i_2 \Rightarrow o_1 = o_2 \wedge q'_1 = q'_2$$

It is *observably nondeterministic* iff

$$\forall \langle q_1, i_1, o_1, q'_1 \rangle, \langle q_2, i_2, o_2, q'_2 \rangle \in T : q_1 = q_2 \wedge i_1 = i_2 \wedge o_1 = o_2 \Rightarrow q'_1 = q'_2$$

Note that observable nondeterminism is equivalent to Assumption 3 above. Deterministic and observably nondeterministic automata allow writing the set of transitions as a (partial) transition function δ of the set of transitions with $\delta : Q \times I \rightarrow O \times Q$ and $\delta : Q \times I \times O \rightarrow Q$ respectively.

The definition of IO automata follows the behavior of Mealy machines, where the output of a transition depends on the source state, the target state and the provided input symbol. This is in contrast to Moore behavior, where the output depends on the target state alone:

Definition 3. An IO automaton $\mathcal{M} = \langle Q, q_0, I, O, T \rangle$ has **Moore behavior** iff

$$\forall \langle q_1, i_1, o_1, q'_1 \rangle, \langle q_2, i_2, o_2, q'_2 \rangle \in T : q'_1 = q'_2 \Rightarrow o_1 = o_2$$

A Moore machine can thus be seen as a deterministic IO automaton $\mathcal{M} = \langle Q, q_0, I, O, T \rangle$ with Moore behavior that emits the output o_0 associated with q_0 prior to handling the first input. Informally, we treat the emission of the initial output as the result of a transition $\langle \epsilon, \epsilon, o_0, q_0 \rangle \in T$.

When learning probabilistic automaton types, not only the possibility of a transition happening needs to be considered, but also its probability. This prompts the use of IO Frequency Automata (IOFA), which augment IOA with an observation count for transitions.

Definition 4. An **IO frequency automaton** is a tuple $\mathcal{M} = \langle Q, q_0, I, O, \delta, \nu \rangle$, where $\langle Q, q_0, I, O, \delta \rangle$ is an observably nondeterministic IO automaton and $\nu : Q \times I \times O \rightarrow \mathbb{N}$ is the frequency function.

The intuition behind $\nu(q, i, o) = n$ is that the transition from q on input i emitting o has been observed n times. We use IOFA as the common internal representation from which all supported automaton types can be extracted. This allows us to use a single implementation which is shared across all supported automaton types. Additional restrictions are required during the merging process to enforce the structural form of those automaton types and are covered in Section 2.3.

2.2 State Merging in the Red-Blue Framework

This section, gives a brief overview over state merging algorithms and the red-blue framework. For a more detailed account consider [10] and [15] respectively.

State-merging algorithms are one of the main categories of passive automata learning algorithms. The input is given as a collection of traces, where each trace is a sequence of input-output pairs. The basic structure of state-merging algorithms consists of the following steps. First, a simple automaton representation of the provided traces is created, commonly using a so-called Prefix Tree Automaton (PTA). The PTA contains a state for every prefix in the input data. Grouping the behavior of traces with shared prefixes is admissible because we assume observable nondeterminism and results in a tree-shaped IOFA that describes exactly the observed traces and no other behavior. In the main loop of

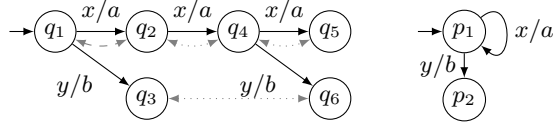


Fig. 1: Example of a PTA without frequencies (left) and the partitioning resulting from merging states q_1 and q_2 (right).

the algorithm, the automaton is generalized by iteratively merging pairs of states until no more merges are possible. We use the term *merge candidate* to refer to a pair of states that is considered for merging. Merging two states q_1 and q_2 results in a state that exhibits the behavior of both states. If both q_1 and q_2 have a transition with input i and output o leading to states q'_1 and q'_2 respectively, the assumption of observable nondeterminism implies that q'_1 and q'_2 must be the same state. Therefore, merging q_1 and q_2 also requires merging q'_1 and q'_2 . This is called an *implied merge*. Together, the initial merging of q_1 and q_2 and its implied merges results in a *partitioning* of the state space into sets of (transitively) merged states. The criteria for selecting potential merge candidates and evaluating their compatibility and quality are specific to the used algorithm.

Example 1 (PTAs and Merging). The left side of Figure 1 shows the PTA for the three traces $\langle x, a \rangle, \langle x, a \rangle, \langle x, a \rangle$, $\langle x, a \rangle, \langle x, a \rangle, \langle y, b \rangle$ and $\langle y, b \rangle$. Trying to merge q_1 and q_2 (dashed line) will lead to the implied merge of q_2 and q_4 (dotted line), which in turn requires merging not only q_4 and q_5 but also q_3 and q_6 since q_1 and q_4 are in the same partition. Applying the merge results in the automaton shown on the right where $p_1 = \{q_1, q_2, q_4, q_5\}$ and $p_2 = \{q_3, q_6\}$.

Algorithm 1 gives a high level description of the red-blue framework. The algorithm is parameterized by a scoring function *score* and an order relation over states $<$. The red-blue framework restricts which pairs of states are considered for merging in each iteration of the algorithm in order to reduce the number of compatibility calculations. This is achieved by dynamically dividing the nodes into three categories throughout the execution of the algorithm. The set of red states contains states that are considered mutually distinct. It is initialized to consist only of the root node of the PTA. The set of blue states is defined as the set of all states reachable from red states in a single transition that are not themselves red. The set of white states consists of all states that are neither red nor blue. In each iteration of the algorithm, only pairs consisting of a red and a blue state are considered for merging. If a blue state is incompatible with all red states, it is promoted to a red state. Otherwise the merge candidate with the best performing score according to an algorithm-specific function is chosen as the next intermediate model. For both promotion and merging, if there are several eligible candidates, ties are broken using a specified node order⁴. When all states are red, no more merges are possible and the final model is returned.

⁴ Omitted in line 12 for brevity.

Algorithm 1 State Merging in the Red-Blue Framework.

Input: Collection of traces $\mathcal{TR}$
Parameters: Scoring function $score$, state order $<$
Output: Automaton model $\mathcal{M}$
Algorithm:

```

1:  $\mathcal{M} \leftarrow$  PTA created from  $\mathcal{TR}$ 
2:  $Q_r \leftarrow \{q_0^{\mathcal{M}}\}$  ▷ Initialize red states to initial state
3: loop
4:    $Q_b \leftarrow$  set of blue states derived from  $Q_r$ 
5:   if  $Q_b = \emptyset$  then
6:     return  $\mathcal{M}$  ▷ Terminate if no more merge candidates are available
7:   end if
8:    $S \leftarrow \{b \in Q_b \mid \forall r \in Q_r : score(\mathcal{M}, r, b) = -\infty\}$  ▷ Unmergeable blue states
9:   if  $S \neq \emptyset$  then
10:     $Q_r \leftarrow Q_r \cup \{\min_{<} S\}$  ▷ Promote minimal unmergeable blue state
11:   else
12:     $\langle r, b \rangle \leftarrow \operatorname{argmax}_{\langle r, b \rangle \in Q_r \times Q_b} score(\mathcal{M}, r, b)$  ▷ Select best merge wrt.  $score()$ 
13:     $\mathcal{M} \leftarrow merge(\mathcal{M}, r, b)$  ▷ Update model
14:   end if
15: end loop
    
```

Example 2 (Red-Blue Framework). Consider a slightly modified PTA compared to Example 1, where the transition from q_4 to q_6 on input y emits a instead of b . Figure 2 shows the progression from the PTA (left) to the final automaton (right) when applying the red-blue framework for learning a deterministic Mealy machine from the provided data. Nodes are ordered according to the shortlex order of their prefixes and labeled q_1 to q_6 accordingly. States q_1 and q_2 can not be merged due to the transitions on input y from states q_1 and q_4 emitting different outputs. The state q_2 is promoted since it is the minimal blue node and incompatible with the only red state q_1 . Due to the order of nodes, q_1 and q_3 are merged before merging q_2 and q_4 . As a result of the latter merge, q_6 becomes a child of q_2 and is finally merged with q_1 .

2.3 Enforcing Automaton Structure

In this section, we describe how the automaton structure of the different supported automaton types is enforced in the internal representation. We categorize the automaton types supported by AALpy in two different ways: (1) output behavior and (2) transition behavior. The output behavior of states may depend on both inputs and the current state, like in Mealy machines, or on states alone, like in Moore machines. Transition behavior may be deterministic, nondeterministic or probabilistic. Taking the product of these two categorization schemes (1 and 2) results in six different automaton types also supported in AALpy. AALpy also supports two additional types for DFAs and discrete time Markov chains. However, these two types are subsumed by the other types. DFAs can be seen as (deterministic) Moore machines with binary output, representing accept and

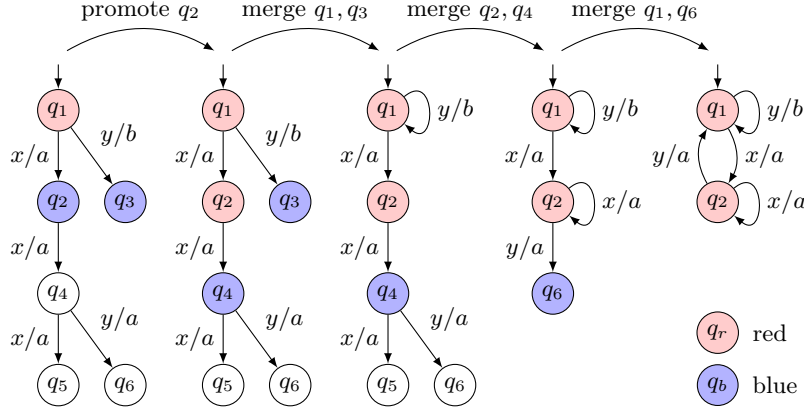


Fig. 2: Intermediate steps of state merging in the red-blue framework.

Table 1: Combination of output behavior (rows) and transition behavior (columns) and the respective automaton classes in AALpy.

	Deterministic	Nondeterministic	Probabilistic
Moore	MooreMachine, Dfa	NDMooreMachine	Mdp, MarkovChain
Mealy	MealyMachine	Onfsm	StochasticMealyMachine

reject. Markov chains have the same semantics as probabilistic Moore machines with a single input that represents the passage of time. Table 1 shows the different combination of behavior types and the respective classes in AALpy.

Output Behavior. Mealy output behavior is the default for IOFAs and thus does not need to be enforced. This is not the case for Moore behavior. However, since each PTA has only one predecessor, Moore behavior (Def. 3) is ensured at the beginning of the state-merging algorithm. The Moore property can be turned into an invariant of state-merging by strengthening the local compatibility criterion during merging such that merges which lead to a violation of the Moore property are rejected. Consequently, Moore behavior of the final automaton is ensured.

Transition Behavior. If deterministic transition behavior is given for a PTA, we can enforce it in the final automaton analogously to Moore behavior by rejecting merge candidates that lead to nondeterministic behavior (Def. 2). Similar to Mealy behavior, nondeterministic transition behavior is the default for IOFA. To extract an automaton with probabilistic transition behavior from an IOFA $\langle Q, q_0, I, O, \delta, \nu \rangle$, the frequency function needs to be normalized. The probability distribution $p_{q,i}(o, q')$ describing the transitions originating from state q on input i to state q' emitting output o is given as

$$p_{q,i}(o, q') = \begin{cases} \nu(q, i, o) / \sum_{o' \in O} \nu(q, i, o') & \text{if } \delta(q, i, o) = q' \\ 0 & \text{otherwise} \end{cases}$$

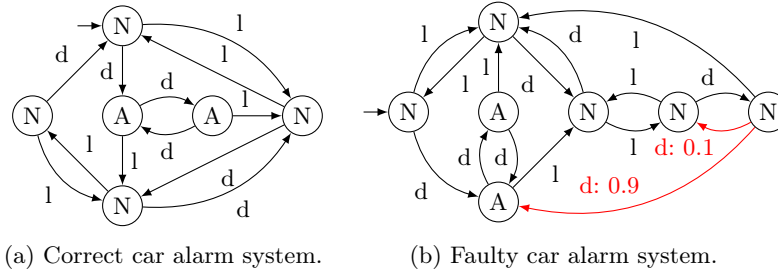


Fig. 3: Car alarm systems with deterministic (a) and probabilistic (b) behavior.

3 Usage

3.1 Defining and Running State-Merging Algorithms

To define a state-merging algorithm, the `GeneralizedStateMerging` class is instantiated. Its most important parameters are:

- output_behavior:** Defines whether an automaton with Moore or Mealy behavior should be learned and adds corresponding restrictions for local compatibility. Possible values are "moore" and "mealy".
- transition_behavior:** Defines whether the learned automaton is assumed to be deterministic, nondeterministic or probabilistic and adds corresponding restrictions for local compatibility. The values are "deterministic", "nondeterministic" and "stochastic" respectively.
- score_calc:** An object of type `ScoreCalculation`, which governs the details of local compatibility and score calculation.

The defined algorithm can be executed using the object's `run` method, which requires a single argument representing the collection of available traces. Each trace is expected to be a sequence of input-output pairs. Algorithms with Moore behavior also expect the traces to be prepended with the initial output. Other input formats are discussed below. Depending on the combination of the arguments `output_behavior` and `transition_behavior`, the `run` method determines what kind of automaton is extracted from the internal representation.

Example 3. Consider a car which can be locked/unlocked (l) and its doors opened/closed (d). The car also has an alarm, which triggers when the car is opened in a locked state and can only be disabled by unlocking the car. Figure 3a shows a Moore machine that models such a car alarm system. Depending on its state, its output is either alarm (A) or no alarm (N). Assume we do not know the real model and want to extract a model from recorded execution traces using RPNI [20], a well-known automata learning algorithm for DFAs which has been extended to Moore and Mealy machines in learning frameworks such as LearnLib [11] and AALpy. Assuming the variable `data` contains known traces from the car alarm system, a Moore machine can be learned using the following code.

```

1 data = [{"N", ("d", "A"), ("d", "A"), ("l", "N"), ("d", "N")}, ... ]
2
3 alg = GeneralizedStateMerging(output_behavior="moore",
4                               transition_behavior="deterministic")
5 model = alg.run(data)

```

No `ScoreCalculation` is provided, since RPNI greedily accepts possible merges without scoring and determinism is automatically enforced when specifying deterministic transition behavior. Mealy machines can be learned simply by setting `output_behavior="mealy"`, assuming `data` has the correct format.

The `ScoreCalculation` class can be instantiated with two parameters, which control local compatibility criteria and scoring. Those parameters are functions operating on the `GsmNode` class, which represents automaton states of IOFA.

`local_compatibility` is a function that takes two `GsmNode` objects and returns whether they are locally compatible. The first argument corresponds to the partition created up to that point and the second argument to the state to be added to the partition. Incompatibility of any two tested nodes leads to immediate rejection of the merge candidate.

`score_function` is a function that takes a dictionary mapping states prior to merging to the state describing the behavior of their associated partition. The function should return some form of score. The values `True` and `False` correspond to a score of positive and negative infinity and leads to immediate acceptance and rejection of the merge respectively.

Example 4. The Evidence Driven State Merging (EDSM) algorithm [15] is similar to RPNI, except that it also uses scoring in order to make better informed decisions. The score of a merge candidate depends on the amount of evidence available supporting the merge. This is quantified as the number of local compatibility checks triggered by merging the two states.

```

1 def EDSM_score(part : Dict[GsmNode, GsmNode]):
2     nr_partitions = len(set(part.values()))
3     nr_merged = len(part)
4     return nr_merged - nr_partitions
5
6 score = ScoreCalculation(score_function=EDSM_score)
7 alg = GeneralizedStateMerging(score_calc=score)

```

An example implementing the IOAlergia algorithm [16] for probabilistic systems such as the faulty car alarm depicted in Figure 3b using a local compatibility criterion can be found in the appendix (Ex. 5).

The `ScoreCalculation` class can also be subclassed to create stateful score functions, which accumulate information from the local compatibility checks as the partitioning is created. In addition to the two methods described above, stateful implementations may override the method `reset` which is used to reset

the internal state between score calculations. An example is provided in the appendix (Ex. 7).

Apart from the options listed above, `GeneralizedStateMerging` also has some minor parameters that change details in the red-blue framework. The options `pta_processing` and `postprocessing` allow to manipulate the internal representation before and after state merging. They can be set to arbitrary functions mapping one `GsmNode` to another and are applied to the initial state. Setting the flags `eval_compat_on_pta` and `eval_compat_on_futures` changes the way how local compatibility is evaluated, which is required e.g. to implement IOAlergia. Using the former causes the algorithm to use the original PTA for evaluating compatibility rather than the current automaton. The latter restricts compatibility checks to common futures, rather than computing it with respect to the partition implied by the merge candidate. The parameter `node_order` allows to specify the order in which merge candidates are evaluated. This is especially relevant for algorithms which greedily accept the first feasible merge without the use of scores. The default is the shortlex order of the prefix. The flag `consider_only_min_blue` further restricts the set of merge candidates to pairs of red states with the minimal blue node according to the node order. Finally, setting the flag `depth_first` causes the algorithm to search for violations of the local compatibility criterion in a depth first manner rather than breath first.

3.2 Utilities

The `run` method of the `GeneralizedStateMerging` class also has optional parameters that facilitate the analysis of algorithms. They can be used when experimenting with new algorithms or to better understand under which circumstances existing algorithms fail.

The parameter `convert` can be set to `False` to avoid converting the internal representation to the specific automaton type and causes `run` to return a `GsmNode` object.

The parameter `instrumentation` can be used to instrument the red-blue framework using an `Instrumentation` object that provides the option for callbacks at important points (merging, promotion, etc.).

Apart from the input format of input-output traces, other input formats are supported depending on the setting. In DFA inference, the training data is usually not prefix-closed. In this case, the input data can be represented as a set of labeled input sequences, where each element is a tuple consisting of an input sequence and a single output corresponding to the observation after applying the input sequence. For probabilistic models without input-output behavior, we support data specified as sequences of observations only. While the data format is usually detected automatically, it can be explicitly specified using the `data_format` argument of the `run` method.

The `visualize` method of the `GsmNode` class provides highly customizable visualization of automata based on the dot format of GraphViz [8]. For example,

this could be used to highlight specific aspects of a model such as states that are locally deterministic, or to visualize incorrect merges during the design phase of new algorithms for debugging purposes.

4 Concluding Remarks

There are several software packages for automata learning. LearnLib [11] is very similar to AALpy in general. It mainly provides active automata learning for deterministic IO automata with a few passive algorithms. While its focus is more narrow, LearnLib provides more algorithms in its domain. The same holds true for jajapy [24], which is mainly concerned with passive learning of probabilistic models. The library libalf [6] falls in a similar category, but seems to be no longer actively developed. FlexFringe [28] is a C++ implementation of the red-blue framework and thus closely related to the presented extension of AALpy. A major difference is that FlexFringe does not support IO behavior out of the box. Supporting IO behavior requires a workaround, especially in the case of observably nondeterministic systems. On the other hand, FlexFringe offers more configuration options. A particularly noteworthy feature is the option to attach custom data to states which is exposed to the compatibility and scoring functions. The tools RALib [7], Tomte [1] and Mint [30] all deal with learning of Extended Finite State Machines in some form, which is not covered in AALpy.

We extended the passive automata learning capabilities of AALpy by adding a general implementation of the red-blue framework [15]. This implementation is general enough to cover state-merging algorithms for various types of IO automata which may exhibit deterministic, (observably) nondeterministic or probabilistic transition behavior and Mealy- or Moore-style output behavior. While the main focus was to provide a method that is general yet simple to use rather than performance, the provided implementation is sufficiently fast for practical applications. For example, we learned DFAs with well over thousand states from several hundred thousand traces in only a few minutes using the `pypy` interpreter on an ordinary laptop. Still, when comparing the RPNI implementations of LearnLib and our framework using the training data from the Abbadingo competition [15], our implementation is slower than LearnLib by a factor of 7.24. For the more complex (i.e. time consuming) Abbadingo tasks, the difference is less pronounced with a factor of 6.3. This constitutes a significant but not prohibitive difference. We assume the main causes for the difference to be (1) the lower performance of Python compared to Java and (2) our more general implementation supporting nondeterministic transitions, which is not required for RPNI and thus overhead. If performance is an issue, a dedicated implementation in a low level language is of course preferable. This is not our intention for AALpy and this extension. In some cases, the implementation abstracts the algorithm based on the parameters to reduce the overhead due to the generality of the red-blue framework. All in all, this results in a versatile state-merging tool which we expect to be useful to both researchers and industry.

In the future, we want to use the new capabilities of AALpy to explore combinations of active and passive algorithms. This can be particularly interesting when domain knowledge is available that can be used to efficiently generate traces with interesting (e.g. safety critical) behavior. We also aim to use the provided framework to explore how domain knowledge about systems can be injected into existing algorithms, which can be helpful in low data scenarios [4].

Acknowledgements. This work has received funding from the AIDORt project (grant agreement No 101007350) from the ECSEL Joint Undertaking (JU). The JU receives support from the European Union’s Horizon 2020 research and innovation programs and Sweden, Austria, Czech Republic, Finland, France, Italy, and Spain.

The research leading to these results has received funding from the Transformative AI-Assisted Testing in Industrial Mobile Robotics (TASTE) project (project number FO999911053) from the ICT of the Future program. ICT of the Future is a research, technology and innovation funding program of the Republic of Austria, Ministry of Climate Action.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Appendix

This appendix provides a collection of examples which illustrate how state-merging algorithms can be defined and executed in our tool. The examples mostly cover algorithms for learning probabilistic models. At the end, we compare the implemented algorithms in a low-data scenario. The resulting automata are shown in Figure 4.

Example 5. Consider the car alarm system from Example 3. A faulty car alarm system might fail to register a closing door when the car is locked but open with a failure rate of 10%. A probabilistic model of the faulty system is shown in Figure 3b with the probabilistic transitions marked in red. Traces from this system may show nondeterministic behavior, which causes RPNI to fail. We can learn a probabilistic model using IOAlergia [16], an algorithm for learning automata with Moore-style output behavior and probabilistic transition behavior. For each input which is enabled for the two states in consideration, the compatibility function of IOAlergia checks whether the respective distributions of outputs are similar enough, considering the amount of available data. This check is based on the Hoeffding-bound, for which we will use the stub `hoeffding_compat` in the code. The method has a single parameter ϵ , which determines, how aggressively the algorithm merges states. The algorithm only evaluates the compatibility on the shared futures of the merge candidate and not on the resulting partitioning. For statistical reasons it also does not evaluate the compatibility on the intermediate hypothesis, but on the PTA. For each transition, the observation counts of the corresponding transition in the PTA can be accessed using the field `original_count`.

```

1 def ioalergia_compat(a: GsmNode, b: GsmNode):
2     transition_dummy = TransitionInfo(None, 0, None, 0)
3     # iterate over inputs that are common to both states
4     in_iter = intersection_iterator(a.transitions, b.transitions)
5     for in_sym, a_trans, b_trans in in_iter:
6         a_tot = sum(x.original_count for x in a_trans.values())
7         b_tot = sum(x.original_count for x in b_trans.values())
8         # iterate over outputs that appear in either distribution
9         out_iter = union_iterator(a_trans, b_trans, transition_dummy)
10        for out_sym, a_trans, b_trans in out_iter:
11            a_cnt = a_trans.original_count
12            b_cnt = b_trans.original_count
13            if not hoeffding_compat(a_cnt, a_tot, b_cnt, b_tot,  $\epsilon$ ):
14                return False
15    return True
16
17 alg = GeneralizedStateMerging(transition_behavior="stochastic",
18     compatibility_on_pta=True, compatibility_on_futures=True,
19     score_calc=ScoreCalculation(ioalergia_compat))

```

Example 6. In this example, we create a modified version of IOAlergia (see Ex. 5). In this version, merge candidates are evaluated by determining local compatibility between each state q prior to merging and the state corresponding to the partition associated with q after merging.⁵ This transformation of a local compatibility function to a score function is implemented in the helper function `local_to_global_compatibility`. Additionally, `original_count` needs to be exchanged with `count` in `ioalergia_compat` to account for the actual frequencies in the current model.

```

1 compat = local_to_global_compatibility(ioalergia_compat)
2 alg = GeneralizedStateMerging(transition_behavior="stochastic",
3     score_calc=ScoreCalculation(score_function=compat))

```

Example 7. As EDSM scoring is known to be a good heuristic for passive learning, it is reasonable to also apply it in the probabilistic case, e.g. on top of IOAlergia. In most cases, reusing scores is very easy in our framework. However, combining IOAlergia and EDSM as defined above overestimates the evidence. This is because the score function is always evaluated on the resulting partitioning whereas IOAlergia only evaluates common futures on the PTA. However, a stateful `ScoreCalculation` object can keep track of the number of local compatibility evaluations as shown below. Assuming a version of `ioalergia_compat` (see Ex. 5) with an additional parameter for ϵ , a dedicated `ScoreCalculation` class also makes instantiating algorithms with different parameters more natural for programmers not used to partial application of functions.

⁵ Note that this invalidates the assumption of the statistical test because samples are not independent anymore.

```

1 class IOAlergiaWithEDSM(ScoreCalculation):
2     def __init__(self, epsilon):
3         super().__init__()
4         self.epsilon = epsilon
5         self.evidence = 0
6
7     def reset(self):
8         self.evidence = 0
9
10    def local_compatibility(self, a: GsmNode, b: GsmNode):
11        self.evidence += 1
12        return ioalergia_compat(a, b, self.epsilon)
13
14    def score_function(self, part: dict[GsmNode, GsmNode]):
15        return self.evidence
16
17 score = IOAlergiaWithEDSM( $\epsilon$ )

```

Example 8. In this example, we show how domain knowledge can be incorporated into learning algorithm. Considering the car alarm system, we know that there is a underlying physical state of the car, which can be any combination of locked / unlocked and open / closed. This state can be observed from the inputs by checking the parity of lock actions and door actions respectively. Since all configurations of the physical state influence the behavior of the car alarm system, we can assume that each state in the learned model relates to exactly one physical state. In other words, states with different physical states should not be merged. This can be formalized as a local compatibility criterion and combined with the compatibility criterion of IOAlergia in a straight-forward manner.

```

1 def car_alarm_parity_compat(a: GsmNode, b: GsmNode):
2     def get_parity(node: GsmNode):
3         pref = node.get_prefix()
4         parity = []
5         for key in ["l", "d"]:
6             val = sum(in_s == key for in_s, out_s in pref) % 2
7             parity.append(val)
8         return parity
9     return get_parity(a) == get_parity(b)
10
11 def local_compat(a: GsmNode, b: GsmNode, epsilon):
12     parity = car_alarm_parity_compat(a, b)
13     ioa = ioalergia_compat(a, b, epsilon)
14     return parity and ioa

```

Example 9. In this example we sketch an algorithm for learning deterministic systems in the presence of noisy (i.e. mislabeled) data. We assume knowledge of an estimate of the per-step mislabeling rate `error_rate` and uniformly distributed noise. The quality of a merge candidate can be assessed by calculating

how unlikely the resulting partitioning is. This is done by looping over all modified states and inputs while keeping track of the number of observations that do not match the most frequent observation as well as the total number of observations. From those two numbers and the per-step mislabeling probability, the probability p_{obs} of observing at least as many mismatches is calculated using a binomial distribution. The merge is rejected by returning `False` if the resulting probability is below a predefined `threshold`. Note that this procedure is a non-local compatibility criterion and is thus implemented as a scoring function. The probability p_{obs} is used as score, favoring merges with fewer inconsistencies. This could be extended by also considering some form of evidence similar to EDSM.

```

1 from scipy.stats import binom
2 def get_main_output(trans: Dict[Any, TransitionInfo]):
3     return max(trans.items(), key=lambda x: x[1].count)[0]
4
5 def nd_score(part: Dict[GsmNode, GsmNode], error_rate, threshold):
6     mismatches = 0
7     total_count = 0
8     for node in set(part.values()):
9         for in_sym, trans in node.transitions.items():
10             main_out = get_main_output(trans)
11             for out_sym, info in trans.items():
12                 if out_sym != main_out:
13                     mismatches += info.count
14                     total_count += info.count
15
16     prob = 1-binom.cdf(mismatches-1, total_count, error_rate)
17     if prob < threshold:
18         return False
19     return prob

```

A post-processing step is used for removing non-dominant transitions, resulting in a deterministic automaton. However, specifying the transition behavior as deterministic would enforce determinism during runtime, which would reduce the algorithm to RPNI. Thus, we manually convert the resulting model to a deterministic automaton formalism.

```

1 def postprocess(root: GsmNode):
2     for node in root.get_all_nodes():
3         for in_sym, trans in node.transitions.items():
4             output = get_main_output(trans)
5             node.transitions[in_sym] = {output: trans[output]}
6     return root
7
8 output_behavior = "moore"
9 score = lambda part: nd_score(part, error_rate, threshold)
10 score = ScoreCalculation(score_function=score)
11 alg = GeneralizedStateMerging(transition_behavior="nondeterministic",
12     score_calc=score, postprocessing=postprocess)

```

```

13 internal_model = alg.run(data, convert=False)
14 model = internal_model.to_automaton(output_behavior, "deterministic")

```

Comparison of Examples

Given enough data, the algorithms from Example 5, 6, 7 and 8 can learn models of the faulty car alarm system which have the correct structure and correct transition probabilities up to sampling error. To give a rough comparison regarding sample efficiency, we run the examples on a limited data set. In particular, we use 200 samples with lengths chosen uniformly from the range $[10, 20]$. The applied inputs are also chosen randomly. For Alergia-based algorithms, we use the default value for ϵ , which is 0.05. We also show the results for the algorithm from Example 9. For this algorithm, we roughly estimate the error frequency across all states to be 1% and use a 5% significance threshold. Figure 4 shows the learned models. The result of IOAlergia, shown in Figure 4a, is off by only one state. However, it has been merged too aggressively, which is a known problem in low data scenarios. Unsurprisingly, using the algorithm from Example 6 does not improve the result (see Fig. 4b). While the result achieved by Example 7 (see Fig. 4c) better approximates the true structure, it still does not reflect the abstract state space formed by the combination of the locking status and door status. This is evident from the self-transition in the initial state. However, combining IOAlergia with both EDSM and evaluation on the partition results in a model, shown in Figure 4d, that contains only a single mistake: the faulty armed state has been merged with the corresponding state in which the system works correctly. The model therefore contains less information on how the fault can be reproduced. The algorithm based on domain knowledge (Ex. 8) yields the same result (see Fig. 4e). The algorithm for learning deterministic models in the presence of noise correctly learns the model of the correct car alarm system (see Fig. 4f).

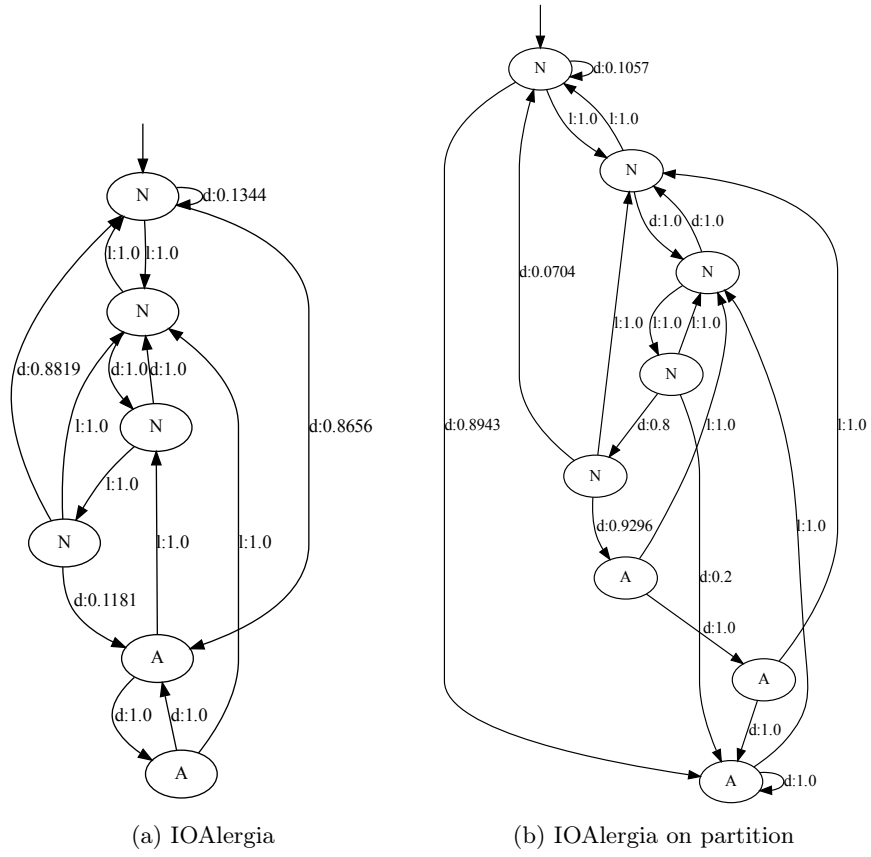


Fig. 4: Learned models of the faulty car alarm system using different algorithms

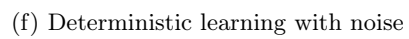
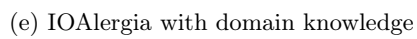
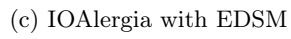


Fig. 4: Learned models of the faulty car alarm system using different algorithms

References

1. Aarts, F., Heidarian, F., Kuppens, H., Olsen, P., Vaandrager, F.W.: Automata learning through counterexample guided abstraction refinement. In: Giannakopoulou, D., Méry, D. (eds.) FM 2012: Formal Methods - 18th International Symposium, Paris, France, August 27-31, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7436, pp. 10–27. Springer (2012). https://doi.org/10.1007/978-3-642-32759-9_4
2. Aichernig, B.K., Tappler, M.: Probabilistic black-box reachability checking (extended version). *Formal Methods Syst. Des.* **54**(3), 416–448 (2019). <https://doi.org/10.1007/S10703-019-00333-0>, <https://doi.org/10.1007/s10703-019-00333-0>
3. Ayoughi, N., Nejati, S., Sabetzadeh, M., Saavedra, P.: Enhancing automata learning with statistical machine learning: A network security case study. In: Egyed, A., Wimmer, M., Chechik, M., Combemale, B. (eds.) Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, MODELS 2024, Linz, Austria, September 22-27, 2024. pp. 172–182. ACM (2024). <https://doi.org/10.1145/3640310.3674087>
4. von Berg, B., Aichernig, B.K., Rindler, M., Stern, D., Tappler, M.: Hierarchical learning of generative automaton models from sequential data. In: Madeira, A., Knapp, A. (eds.) Software Engineering and Formal Methods - 22nd International Conference, SEFM 2024, Aveiro, Portugal, November 6-8, 2024, Proceedings. Lecture Notes in Computer Science, vol. 15280, pp. 215–233. Springer (2024). https://doi.org/10.1007/978-3-031-77382-2_13
5. Bollig, B., Függer, M., Nowak, T.: A framework for streaming event-log prediction in business processes. *CoRR* **abs/2412.16032** (2024). <https://doi.org/10.48550/ARXIV.2412.16032>
6. Bollig, B., Katoen, J., Kern, C., Leucker, M., Neider, D., Piegdon, D.R.: libalf: The automata learning framework. In: Touili, T., Cook, B., Jackson, P.B. (eds.) Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. Lecture Notes in Computer Science, vol. 6174, pp. 360–364. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_32
7. Cassel, S., Falk, H., Jonsson, B.: RALib : A LearnLib extension for inferring EFSSMs. In: International Workshop on Design and Implementation of Formal Tools and Systems, DIFTS 2015, Austin, TX, USA, September 26-27 (2015), <https://api.semanticscholar.org/CorpusID:12915995>
8. Ellson, J., Gansner, E.R., Koutsofios, E., North, S.C., Woodhull, G.: Graphviz - open source graph drawing tools. In: Mutzel, P., Jünger, M., Leipert, S. (eds.) Graph Drawing, 9th International Symposium, GD 2001 Vienna, Austria, September 23-26, 2001, Revised Papers. Lecture Notes in Computer Science, vol. 2265, pp. 483–484. Springer (2001). https://doi.org/10.1007/3-540-45848-4_57
9. Ganty, P.: Learning the state machine behind a modal text editor: The (neo)vim case study. In: Neele, T., Wijs, A. (eds.) Model Checking Software - 30th International Symposium, SPIN 2024, Luxembourg City, Luxembourg, April 8-9, 2024, Proceedings. Lecture Notes in Computer Science, vol. 14624, pp. 167–175. Springer (2024). https://doi.org/10.1007/978-3-031-66149-5_9
10. de la Higuera, C.: Grammatical Inference: Learning Automata and Grammars. Cambridge University Press (2010). <https://doi.org/10.1017/CB09781139194655>

11. Isberner, M., Howar, F., Steffen, B.: The open-source learnlib - A framework for active automata learning. In: Kroening, D., Pasareanu, C.S. (eds.) *Computer Aided Verification - 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 9206, pp. 487–495. Springer (2015). https://doi.org/10.1007/978-3-319-21690-4_32
12. Junges, S., Rot, J.: Learning language intersections. In: Jansen, N., Stoelinga, M., van den Bos, P. (eds.) *A Journey from Process Algebra via Timed Automata to Model Learning - Essays Dedicated to Frits Vaandrager on the Occasion of His 60th Birthday. Lecture Notes in Computer Science*, vol. 13560, pp. 371–381. Springer (2022). https://doi.org/10.1007/978-3-031-15629-8_20
13. Kobialka, P., Pferscher, A., Bergersen, G.R., Johnsen, E.B., Tarifa, S.L.T.: Stochastic games for user journeys. In: Platzer, A., Rozier, K.Y., Pradella, M., Rossi, M. (eds.) *Formal Methods - 26th International Symposium, FM 2024, Milan, Italy, September 9-13, 2024, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 14934, pp. 167–186. Springer (2024). https://doi.org/10.1007/978-3-031-71177-0_12
14. Kuze, N., Seno, K., Ushio, T.: Learning-based black box checking for k-safety hyperproperties. *Eng. Appl. Artif. Intell.* **126**, 107029 (2023). <https://doi.org/10.1016/J.ENGAPPAI.2023.107029>
15. Lang, K.J., Pearlmutter, B.A., Price, R.A.: Results of the Abbadingo One DFA learning competition and a new evidence-driven state merging algorithm. In: Honavar, V.G., Slutzki, G. (eds.) *Grammatical Inference, 4th International Colloquium, ICGI-98, Ames, Iowa, USA, July 12-14, 1998, Proceedings. Lecture Notes in Computer Science*, vol. 1433, pp. 1–12. Springer (1998). <https://doi.org/10.1007/BFB0054059>
16. Mao, H., Chen, Y., Jaeger, M., Nielsen, T.D., Larsen, K.G., Nielsen, B.: Learning deterministic probabilistic automata from a model checking perspective. *Mach. Learn.* **105**(2), 255–299 (2016). <https://doi.org/10.1007/s10994-016-5565-9>
17. Moddemann, L., Steude, H.S., Diedrich, A., Pill, I., Niggemann, O.: Extracting knowledge using machine learning for anomaly detection and root-cause diagnosis. In: *29th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2024, Padova, Italy, September 10-13, 2024*. pp. 1–8. IEEE (2024). <https://doi.org/10.1109/ETFA61755.2024.10710647>
18. Muskardin, E., Aichernig, B.K., Pill, I., Pferscher, A., Tappler, M.: Aalpy: an active automata learning library. *Innov. Syst. Softw. Eng.* **18**(3), 417–426 (2022). <https://doi.org/10.1007/S11334-022-00449-3>
19. Muskardin, E., Aichernig, B.K., Pill, I., Tappler, M.: Learning finite state models from recurrent neural networks. In: ter Beek, M.H., Monahan, R. (eds.) *Integrated Formal Methods - 17th International Conference, IFM 2022, Lugano, Switzerland, June 7-10, 2022, Proceedings. Lecture Notes in Computer Science*, vol. 13274, pp. 229–248. Springer (2022). https://doi.org/10.1007/978-3-031-07727-2_13
20. Oncina, J., García, P.: Identifying regular languages in polynomial time. In: Bunke, H. (ed.) *Advances in Structural and Syntactic Pattern Recognition. Series in Machine Perception and Artificial Intelligence*, vol. 5, pp. 99–108. World Scientific (1993). https://doi.org/10.1142/9789812797919_0007, https://www.worldscientific.com/doi/abs/10.1142/9789812797919_0007
21. Peled, D., Vardi, M.Y., Yannakakis, M.: Black box checking. In: *International Conference on Protocol Specification, Testing and Verification*. pp. 225–240. Springer (1999)

22. Pferscher, A., Aichernig, B.K.: Fingerprinting bluetooth low energy devices via active automata learning. In: Huisman, M., Pasareanu, C.S., Zhan, N. (eds.) *Formal Methods - 24th International Symposium, FM 2021, Virtual Event, November 20-26, 2021, Proceedings. Lecture Notes in Computer Science*, vol. 13047, pp. 524–542. Springer (2021). https://doi.org/10.1007/978-3-030-90870-6_28
23. Pferscher, A., Aichernig, B.K.: Stateful black-box fuzzing of bluetooth devices using automata learning. In: Deshmukh, J.V., Havelund, K., Perez, I. (eds.) *NASA Formal Methods - 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24-27, 2022, Proceedings. Lecture Notes in Computer Science*, vol. 13260, pp. 373–392. Springer (2022). https://doi.org/10.1007/978-3-031-06773-0_20
24. Reynouard, R., Ingólfssdóttir, A., Bacci, G.: Jajapy: A learning library for stochastic models. In: Jansen, N., Tribastone, M. (eds.) *Quantitative Evaluation of Systems - 20th International Conference, QEST 2023, Antwerp, Belgium, September 20-22, 2023, Proceedings. Lecture Notes in Computer Science*, vol. 14287, pp. 30–46. Springer (2023). https://doi.org/10.1007/978-3-031-43835-6_3
25. Shijubo, J., Waga, M., Suenaga, K.: Probabilistic black-box checking via active MDP learning. *ACM Trans. Embed. Comput. Syst.* **22**(5s), 148:1–148:26 (2023). <https://doi.org/10.1145/3609127>
26. Tappler, M., Pferscher, A., Aichernig, B.K., Könighofer, B.: Learning and repair of deep reinforcement learning policies from fuzz-testing data. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. pp. 6:1–6:13. ACM (2024). <https://doi.org/10.1145/3597503.3623311>
27. Tappler, M., Pranger, S., Könighofer, B., Muskardin, E., Bloem, R., Larsen, K.G.: Automata learning meets shielding. In: Margaria, T., Steffen, B. (eds.) *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles - 11th International Symposium, ISoLA 2022, Rhodes, Greece, October 22-30, 2022, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 13701, pp. 335–359. Springer (2022). https://doi.org/10.1007/978-3-031-19849-6_20
28. Verwer, S., Hammerschmidt, C.A.: Flexfringe: Modeling software behavior by learning probabilistic automata. *CoRR* **abs/2203.16331** (2022). <https://doi.org/10.48550/ARXIV.2203.16331>
29. Verwer, S., de Weerd, M., Witteveen, C.: A likelihood-ratio test for identifying probabilistic deterministic real-time automata from positive data. In: Sempere, J.M., García, P. (eds.) *Grammatical Inference: Theoretical Results and Applications, 10th International Colloquium, ICGI 2010, Valencia, Spain, September 13-16, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6339, pp. 203–216. Springer (2010). https://doi.org/10.1007/978-3-642-15488-1_17
30. Walkinshaw, N., Taylor, R., Derrick, J.: Inferring extended finite state machine models from software executions. *Empir. Softw. Eng.* **21**(3), 811–853 (2016). <https://doi.org/10.1007/S10664-015-9367-7>